

Why does this work? Are there not other functions with the same first five terms in their power series but completely different behavior in the range (say) $2 < x < 10$? Indeed there are. Padé approximation has the uncanny knack of picking the function *you had in mind* from among all the possibilities. *Except when it doesn't!* That is the downside of Padé approximation: It is uncontrolled. There is, in general, no way to tell how accurate it is, or how far out in x it can usefully be extended. It is a powerful, but in the end still mysterious, technique.

Here is the routine that returns a Ratfn rational function object that is the Padé approximant to a set of power series coefficients that you provide. Note that the routine is specialized to the case $M = N$. You can then use the Ratfn object directly as a functor, or else read out its coefficients by hand (§5.1).

Ratfn pade(VecDoub_I &cof)

pade.h

Given cof[0..2*n], the leading terms in the power series expansion of a function, solve the linear Padé equations to return a Ratfn object that embodies a diagonal rational function approximation to the same function.

```
{
    const Doub BIG=1.0e99;
    Int j,k,n=(cof.size()-1)/2;
    Doub sum;
    MatDoub q(n,n),qlu(n,n);
    VecInt indx(n);
    VecDoub x(n),y(n),num(n+1),denom(n+1);
    for (j=0;j<n;j++) {                Set up matrix for solving.
        y[j]=cof[n+j+1];
        for (k=0;k<n;k++) q[j][k]=cof[j-k+n];
    }
    LUdcmp lu(q);                      Solve by LU decomposition and backsubstitution,
    lu.solve(y,x);                      with iterative improvement.
    for (j=0;j<4;j++) lu.mprove(y,x);
    for (k=0;k<n;k++) {                Calculate the remaining coefficients.
        for (sum=cof[k+1],j=0;j<=k;j++) sum -= x[j]*cof[k-j];
        y[k]=sum;
    }
    num[0] = cof[0];
    denom[0] = 1.;
    for (j=0;j<n;j++) {                Copy answers to output.
        num[j+1]=y[j];
        denom[j+1] = -x[j];
    }
    return Ratfn(num,denom);
}
```

CITED REFERENCES AND FURTHER READING:

- Ralston, A. and Wilf, H.S. 1960, *Mathematical Methods for Digital Computers* (New York: Wiley), p. 14.
- Cuyt, A., and Wuytack, L. 1987, *Nonlinear Methods in Numerical Analysis* (Amsterdam: North-Holland), Chapter 2.
- Graves-Morris, P.R. 1979, in *Padé Approximation and Its Applications*, Lecture Notes in Mathematics, vol. 765, L. Wuytack, ed. (Berlin: Springer).[1]

5.13 Rational Chebyshev Approximation

In §5.8 and §5.10 we learned how to find good polynomial approximations to a given function $f(x)$ in a given interval $a \leq x \leq b$. Here, we want to generalize the task to find

good approximations that are rational functions (see §5.1). The reason for doing so is that, for some functions and some intervals, the optimal rational function approximation is able to achieve substantially higher accuracy than the optimal polynomial approximation with the same number of coefficients. This must be weighed against the fact that finding a rational function approximation is not as straightforward as finding a polynomial approximation, which, as we saw, could be done elegantly via Chebyshev polynomials.

Let the desired rational function $R(x)$ have a numerator of degree m and denominator of degree k . Then we have

$$R(x) \equiv \frac{p_0 + p_1x + \cdots + p_mx^m}{1 + q_1x + \cdots + q_kx^k} \approx f(x) \quad \text{for } a \leq x \leq b \quad (5.13.1)$$

The unknown quantities that we need to find are $p_0, \dots, p_m$ and $q_1, \dots, q_k$, that is, $m + k + 1$ quantities in all. Let $r(x)$ denote the deviation of $R(x)$ from $f(x)$, and let r denote its maximum absolute value,

$$r(x) \equiv R(x) - f(x) \quad r \equiv \max_{a \leq x \leq b} |r(x)| \quad (5.13.2)$$

The ideal *minimax* solution would be that choice of p 's and q 's that minimizes r . Obviously there is *some* minimax solution, since r is bounded below by zero. How can we find it, or a reasonable approximation to it?

A first hint is furnished by the following fundamental theorem: If $R(x)$ is nondegenerate (has no common polynomial factors in numerator and denominator), then there is a unique choice of p 's and q 's that minimizes r ; for this choice, $r(x)$ has $m + k + 2$ extrema in $a \leq x \leq b$, all of magnitude r and with alternating sign. (We have omitted some technical assumptions in this theorem. See Ralston [1] for a precise statement.) We thus learn that the situation with rational functions is quite analogous to that for minimax polynomials: In §5.8 we saw that the error term of an n th-order approximation, with $n + 1$ Chebyshev coefficients, was generally dominated by the first neglected Chebyshev term, namely T_{n+1} , which itself has $n + 2$ extrema of equal magnitude and alternating sign. So, here, the number of rational coefficients, $m + k + 1$, plays the same role of the number of polynomial coefficients, $n + 1$.

A different way to see why $r(x)$ should have $m + k + 2$ extrema is to note that $R(x)$ can be made exactly equal to $f(x)$ at any $m + k + 1$ points x_i . Multiplying equation (5.13.1) by its denominator gives the equations

$$p_0 + p_1x_i + \cdots + p_mx_i^m = f(x_i)(1 + q_1x_i + \cdots + q_kx_i^k) \quad i = 0, 1, \dots, m + k \quad (5.13.3)$$

This is a set of $m + k + 1$ linear equations for the unknown p 's and q 's, which can be solved by standard methods (e.g., LU decomposition). If we choose the x_i 's to all be in the interval (a, b) , then there will generically be an extremum between each chosen x_i and x_{i+1} , plus also extrema where the function goes out of the interval at a and b , for a total of $m + k + 2$ extrema. For arbitrary x_i 's, the extrema will not have the same magnitude. The theorem says that, for one particular choice of x_i 's, the magnitudes can be beaten down to the identical, minimal, value of r .

Instead of making $f(x_i)$ and $R(x_i)$ equal at the points x_i , one can instead force the residual $r(x_i)$ to any desired values y_i by solving the linear equations

$$p_0 + p_1x_i + \cdots + p_mx_i^m = [f(x_i) - y_i](1 + q_1x_i + \cdots + q_kx_i^k) \quad i = 0, 1, \dots, m + k \quad (5.13.4)$$

In fact, if the x_i 's are chosen to be the extrema (not the zeros) of the minimax solution, then the equations satisfied will be

$$p_0 + p_1x_i + \cdots + p_mx_i^m = [f(x_i) \pm r](1 + q_1x_i + \cdots + q_kx_i^k) \quad i = 0, 1, \dots, m + k + 1 \quad (5.13.5)$$

where the $\pm$ alternates for the alternating extrema. Notice that equation (5.13.5) is satisfied at $m + k + 2$ extrema, while equation (5.13.4) was satisfied only at $m + k + 1$ arbitrary points. How can this be? The answer is that r in equation (5.13.5) is an additional unknown, so that

the number of both equations and unknowns is $m + k + 2$. True, the set is mildly nonlinear (in r), but in general it is still perfectly soluble by methods that we will develop in Chapter 9.

We thus see that, given only the *locations* of the extrema of the minimax rational function, we can solve for its coefficients and maximum deviation. Additional theorems, leading up to the so-called *Remes algorithms* [1], tell how to converge to these locations by an iterative process. For example, here is a (slightly simplified) statement of *Remes' Second Algorithm*: (1) Find an initial rational function with $m + k + 2$ extrema x_i (not having equal deviation). (2) Solve equation (5.13.5) for new rational coefficients and r . (3) Evaluate the resulting $R(x)$ to find its actual extrema (which will not be the same as the guessed values). (4) Replace each guessed value with the nearest actual extremum of the same sign. (5) Go back to step 2 and iterate to convergence. Under a broad set of assumptions, this method will converge. Ralston [1] fills in the necessary details, including how to find the initial set of x_i 's.

Up to this point, our discussion has been textbook standard. We now reveal ourselves as heretics. We don't much like the elegant Remes algorithm. Its two nested iterations (on r in the nonlinear set 5.13.5, and on the new sets of x_i 's) are finicky and require a lot of special logic for degenerate cases. Even more heretical, we doubt that compulsive searching for the *exactly best*, equal deviation approximation is worth the effort — except perhaps for those few people in the world whose business it is to find optimal approximations that get built into compilers and microcode.

When we use rational function approximation, the goal is usually much more pragmatic: Inside some inner loop we are evaluating some function a zillion times, and we want to speed up its evaluation. Almost never do we need this function to the last bit of machine accuracy. Suppose (heresy!) we use an approximation whose error has $m + k + 2$ extrema whose deviations differ by a factor of 2. The theorems on which the Remes algorithms are based guarantee that the perfect minimax solution will have extrema somewhere within this factor of 2 range — forcing down the higher extrema will cause the lower ones to rise, until all are equal. So our “sloppy” approximation is in fact within a fraction of a least significant bit of the minimax one.

That is good enough for us, especially when we have available a very robust method for finding the so-called “sloppy” approximation. Such a method is the least-squares solution of overdetermined linear equations by singular value decomposition (§2.6 and §15.4). We proceed as follows: First, solve (in the least-squares sense) equation (5.13.3), not just for $m + k + 1$ values of x_i , but for a significantly larger number of x_i 's, spaced approximately like the zeros of a high-order Chebyshev polynomial. This gives an initial guess for $R(x)$. Second, tabulate the resulting deviations, find the mean absolute deviation, call it r , and then solve (again in the least-squares sense) equation (5.13.5) with r fixed and the $\pm$ chosen to be the sign of the observed deviation at each point x_i . Third, repeat the second step a few times.

You can spot some Remes orthodoxy lurking in our algorithm: The equations we solve are trying to bring the deviations not to zero, but rather to plus-or-minus some consistent value. However, we dispense with keeping track of actual extrema, and we solve only linear equations at each stage. One additional trick is to solve a *weighted* least-squares problem, where the weights are chosen to beat down the largest deviations fastest.

Here is a function implementing these ideas. Notice that the only calls to the function `fn` occur in the initial filling of the table `fs`. You could easily modify the code to do this filling outside of the routine. It is not even necessary that your abscissas `xs` be exactly the ones that we use, though the quality of the fit will deteriorate if you do not have several abscissas between each extremum of the (underlying) minimax solution. The function returns a `Ratfn` object that you can subsequently use as a functor, or from which you can extract the stored coefficients.

```
Ratfn ratlsq(Doub fn(const Doub), const Doub a, const Doub b, const Int mm,      ratlsq.h
             const Int kk, Doub &dev)
```

Returns a rational function approximation to the function `fn` in the interval `(a, b)`. Input quantities `mm` and `kk` specify the order of the numerator and denominator, respectively. The maximum absolute deviation of the approximation (insofar as is known) is returned as `dev`.

```
{
    const Int NPFAC=8,MAXIT=5;
    const Doub BIG=1.0e99,PI02=1.570796326794896619;
```

```

Int i,it,j,ncof=mm+kk+1,npt=NPFAC*ncof;
Number of points where function is evaluated, i.e., fineness of the mesh.
Doub devmax,e,hth,power,sum;
VecDoub bb(npt),coff(ncof),ee(npt),fs(npt),wt(npt),xs(npt);
MatDoub u(npt,ncof);
Ratfn ratbest(coff,mm+1,kk+1);
dev=BIG;
for (i=0;i<npt;i++) {
    if (i < (npt/2)-1) {
        hth=PI02*i/(npt-1.0);
        xs[i]=a+(b-a)*SQR(sin(hth));
    } else {
        hth=PI02*(npt-i)/(npt-1.0);
        xs[i]=b-(b-a)*SQR(sin(hth));
    }
    fs[i]=fn(xs[i]);
    wt[i]=1.0;
    ee[i]=1.0;
}
e=0.0;
for (it=0;it<MAXIT;it++) {
    for (i=0;i<npt;i++) {
        power=wt[i];
        bb[i]=power*(fs[i]+SIGN(e,ee[i]));
        Key idea here: Fit to  $fn(x) + e$  where the deviation is positive, to  $fn(x) - e$  where
        it is negative. Then  $e$  is supposed to become an approximation to the equal-ripple
        deviation.
        for (j=0;j<mm+1;j++) {
            u[i][j]=power;
            power *= xs[i];
        }
        power = -bb[i];
        for (j=mm+1;j<ncof;j++) {
            power *= xs[i];
            u[i][j]=power;
        }
    }
    SVD svd(u);
    svd.solve(bb,coff);
    In especially singular or difficult cases, one might here edit the singular values, replacing
    small values by zero in w[0..ncof-1].
    devmax=sum=0.0;
    Ratfn rat(coff,mm+1,kk+1);
    for (j=0;j<npt;j++) {
        ee[j]=rat(xs[j])-fs[j];
        wt[j]=abs(ee[j]);
        sum += wt[j];
        if (wt[j] > devmax) devmax=wt[j];
    }
    e=sum/npt;
    if (devmax <= dev) {
        ratbest = rat;
        dev=devmax;
    }
    cout << " ratlsq iteration= " << it;
    cout << " max error= " << setw(10) << devmax << endl;
}
return ratbest;
}

```

Fill arrays with mesh abscissas and function values.

At each end, use formula that minimizes round-off sensitivity.

In later iterations we will adjust these weights to combat the largest deviations.

Loop over iterations.

Set up the "design matrix" for the least-squares fit.

Singular value decomposition.

Tabulate the deviations and revise the weights.

Use weighting to emphasize most deviant points.

Update e to be the mean absolute deviation.

Save only the best coefficient set found.

Figure 5.13.1 shows the discrepancies for the first five iterations of `ratlsq` when it is applied to find the $m = k = 4$ rational fit to the function $f(x) = \cos x / (1 + e^x)$ in the interval $(0, \pi)$. One sees that after the first iteration, the results are virtually as good as the